DAY 15 – ADVANCED SQL

Answers & Practical SQL Queries | MySQL

Objective: Practice aggregate functions, GROUP BY, HAVING, joins, subqueries, table relationships, indexes, and normalization using a simple Student–Course database.

1. Create the Database and Tables

First create a database. The **students**, **courses**, and **enrollments** tables represent a many-to-many relationship: one student can enroll in many courses, and one course can have many students.

```
CREATE DATABASE day15_advanced_sql;
USE day15_advanced_sql;

CREATE TABLE students (
    student_id INT PRIMARY KEY AUTO_INCREMENT,
    student_name VARCHAR(100) NOT NULL,
    city VARCHAR(50)
);

CREATE TABLE courses (
    course_id INT PRIMARY KEY AUTO_INCREMENT,
    course_name VARCHAR(100) NOT NULL,
    fee DECIMAL(10,2) NOT NULL
);

CREATE TABLE enrollments (
    enrollment_id INT PRIMARY KEY AUTO_INCREMENT,
    student_id INT NOT NULL,
    course_id INT NOT NULL,
    marks DECIMAL(5,2),
    FOREIGN KEY (student_id) REFERENCES students(student_id),
    FOREIGN KEY (course_id) REFERENCES courses(course_id),
    UNIQUE (student_id, course_id)
);
```

2. Insert Sample Records

```
INSERT INTO students (student_name, city) VALUES
('Aarav', 'Pune'),
('Riya', 'Mumbai'),
('Neha', 'Pune'),
('Kabir', 'Nashik');

INSERT INTO courses (course_name, fee) VALUES
('Python', 5000),
('SQL', 4000),
('Web Development', 7000);

INSERT INTO enrollments (student_id, course_id, marks) VALUES
(1, 1, 85),
(1, 2, 90),
(2, 1, 78),
(2, 3, 88),
(3, 2, 95),
(4, 1, 67);
```

3. Aggregate Functions

Aggregate functions perform calculations on multiple rows and return a single value.

COUNT() – Count total students

```
SELECT COUNT(*) AS total_students FROM students;
```

SUM() – Calculate the total course fees

```
SELECT SUM(fee) AS total_course_fees FROM courses;
```

AVG() – Calculate the average marks

```
SELECT AVG(marks) AS average_marks FROM enrollments;
```

MIN() and MAX() – Find the lowest and highest marks

```
SELECT
    MIN(marks) AS lowest_marks,
    MAX(marks) AS highest_marks
FROM enrollments;
```

4. GROUP BY and HAVING

GROUP BY combines rows with the same value. **HAVING** filters grouped results, usually after aggregate calculations.

Count how many students are enrolled in each course:

```
SELECT c.course_name, COUNT(e.student_id) AS total_students
FROM courses c
LEFT JOIN enrollments e ON c.course_id = e.course_id
GROUP BY c.course_id, c.course_name;
```

Show only courses having more than one enrolled student:

```
SELECT c.course_name, COUNT(e.student_id) AS total_students
FROM courses c
JOIN enrollments e ON c.course_id = e.course_id
GROUP BY c.course_id, c.course_name
HAVING COUNT(e.student_id) > 1;
```

5. Joins – Retrieve Student and Course Data

An INNER JOIN returns rows with matching records in both tables. The following query displays each student's enrolled course and marks.

```
SELECT s.student_name, c.course_name, e.marks
FROM students s
INNER JOIN enrollments e ON s.student_id = e.student_id
INNER JOIN courses c ON e.course_id = c.course_id
ORDER BY s.student_id;
```

LEFT JOIN – Display all students, including students who have not enrolled in a course:

```
SELECT s.student_name, c.course_name
FROM students s
LEFT JOIN enrollments e ON s.student_id = e.student_id
LEFT JOIN courses c ON e.course_id = c.course_id;
```

6. Subqueries

A subquery is a query written inside another SQL query. It can be used to compare values with a calculated result.

Find students whose marks are above the overall average:

```
SELECT s.student_name, c.course_name, e.marks
FROM students s
JOIN enrollments e ON s.student_id = e.student_id
JOIN courses c ON e.course_id = c.course_id
WHERE e.marks > (SELECT AVG(marks) FROM enrollments);
```

Find courses whose fee is greater than the average course fee:

```
SELECT course_name, fee
FROM courses
WHERE fee > (SELECT AVG(fee) FROM courses);
```

7. Relationships Between Tables

The database uses primary keys and foreign keys to connect records:

```
students.student_id  (Primary Key)
        |
        |  One student can have many enrollments
        v
enrollments.student_id  (Foreign Key)

courses.course_id  (Primary Key)
        |
        |  One course can have many enrollments
        v
enrollments.course_id  (Foreign Key)
```

Relationship type: Students and Courses have a many-to-many relationship. The enrollments table works as a junction (bridge) table and stores extra information such as marks.

8. Index Basics

An index helps MySQL find rows faster for suitable searches. Indexes can improve read performance, but they use storage and can slow down INSERT, UPDATE, and DELETE operations.

```
-- Create an index on student city
CREATE INDEX idx_students_city ON students(city);

-- View indexes on the students table
SHOW INDEX FROM students;

-- Remove the index when it is no longer required
DROP INDEX idx_students_city ON students;
```

9. Normalization Fundamentals

Normalization organizes data into related tables to reduce duplicate data and avoid update, insertion, and deletion problems.

First Normal Form (1NF): Each column contains a single value; there are no repeating groups. For example, store one course per enrollment row rather than keeping a comma-separated course list in the students table.

Second Normal Form (2NF): The table is in 1NF and every non-key column depends on the whole candidate key, not just part of a composite key. Separating student details and course details from enrollment records helps avoid partial dependencies.

Third Normal Form (3NF): The table is in 2NF and non-key columns do not depend on other non-key columns. Student city is stored with the student, and course fee is stored with the course, rather than repeating those details in every enrollment row.

10. Identify Duplicate Data

Check whether the same student name and city combination appears more than once:

```
SELECT student_name, city, COUNT(*) AS duplicate_count
FROM students
GROUP BY student_name, city
HAVING COUNT(*) > 1;
```

The UNIQUE constraint on (student_id, course_id) in the enrollments table also prevents the same student from being enrolled in the same course more than once.

11. Quick Revision

SQL concept	Purpose
COUNT()	Counts rows or non-NULL values.
SUM()	Adds numeric values.
AVG()	Calculates the average.

SQL concept	Purpose
MIN() / MAX()	Finds the smallest / largest value.
GROUP BY	Groups rows with matching values.
HAVING	Filters grouped results.
JOIN	Combines related rows from tables.
Subquery	A query inside another query.
Primary Key	Uniquely identifies each row.
Foreign Key	Links a row to a key in another table.
Index	Helps speed up suitable data searches.
Normalization	Reduces unnecessary duplication and anomalies.

Conclusion: In this practical, aggregate functions, grouping, joins, subqueries, relationships, indexes, and normalization were practiced using a Student–Course database in MySQL.

Note: Run the statements in order in MySQL Workbench or another MySQL client. The sample records are for practice.